TP #6 : I/O et fichiers

L'objectif de ce TP est d'être une introduction à la manipulation des entrées et sorties (ou "I/O") en Java, en utilisant les classes pour lecture, écriture et manipulation de *streams* (flux de données) entre l'application et des sources des données. Ces classes sont définis dans le paquetage *java.io.**.

1 Encodage de fichiers

Pour tous les fichiers, il existe deux encodages possibles : texte ou binaire. Les deux peuvent stocker la même information, et les deux utilisent des séquences des octets. La différence se trouve en la manière d'interpréter ces séquences : dans un encodage texte ils représentent des caractères, en tant que pour un encodage binaire, ils peuvent représenter des données "personnalisées".

Un **fichier binaire** peut contenir plusieurs types des donnés (texte, image, audio, etc). Ces données peuvent être interprétées par des programmes de soutien, mais apparaissent sous forme de texte tronqués dans un éditeur de texte.

Un **fichier texte** peut contenir que des données textuelles ; ils sont plus restrictives que les fichiers binaires mais ils sont moins susceptibles de se corrompre. Les formats typiques des données textuelles sont les formats .TXT (texte brut) ou le format .RTF (texte enrichi).

2 Le paquetage *java.io*

Les entrées/sorties sont gérées par des classes dont la plupart sont dans le paquetage `java.io`. Ces classes permettent de traiter des flux (stream) de données. Un flux représente une suite d'octets (par exemple un fichier ou une chaîne de caractères) qu'on peut lire (entrée) ou dans lequel on peut écrire (sortie).

Les classes essentielles pour traiter ces flux des données sont décrits ci-dessous. Des diagrammes avec toutes les classes dans le paquetage se trouvent dans cette page.

— `Reader` et `Writer` sont des classes abstraites pour le traitement des données textuelles.
— `FileReader` et `FileWriter` pour lire et écrire des données textuelles dans un fichier texte.
— `InputStream` et `OutputStream` pour les flux d'octets ou flux binaires.

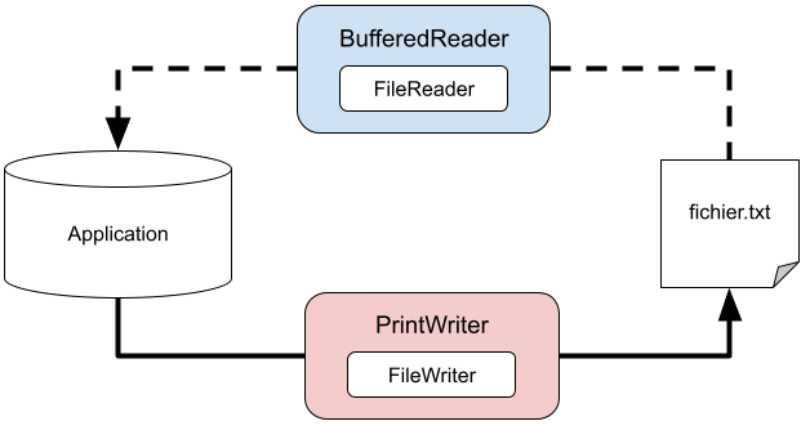
2.1 Traitement des flux textuelles

Lecture : Dans la classe d'extraction `java.io.FileReader`, le constructeur `FileReader(String filename)` ouvre un flux de lecture à partir du fichier indiqué. Si ce fichier n'existe pas ou que le fichier n'est pas valide (c'est un répertoire par exemple), une exception de type `FileNotFoundException` est levée.

La classe de fonctionnalité `java.io.BufferedReader` lit un flux de caractères en les plaçant dans un tampon, ce qui d'une part permet d'éviter de charger d'un seul coup l'intégralité du fichier en mémoire et d'autre part d'accéder directement aux données du fichier à chaque lecture d'un caractère. Le constructeur `BufferedReader(Reader in)` ouvre un flux de lecture bufferisé à partir du flux d'entrée, qui peut provenir du clavier, d'un fichier, etc. Elle possède des méthodes de lecture d'un caractère ou d'une ligne.

Écriture : Dans la classe d'extraction `java.io.FileWriter`, le constructeur `FileWriter(String filename)` permet d'écrire dans le fichier de nom `filename`. S'il n'existe pas, il est créé automatiquement.

La classe de fonctionnalité `java.io.PrintWriter` facilite l'écriture de différents types de données dans un flux de caractères, en fonction de leur type. Elle dispose d'un constructeur recevant en argument un objet de type Writer. La classe PrintWriter possède les méthodes `print()` et `println()` qui, à la manière de Scanner, permettent d'écrire des données primitives dans un flux de caractères.



Exemple : Considérons une application qui lit un fichier texte puis il ajoute des données quelconques :

```
import java.lang.io.*;

public class Application {
    public static void main(String[] args) throws IOException {
        // Lecture
        BufferedReader in = null;
        try {
            in = new BufferedReader(new FileReader("fichier.txt"));
            String ligne = null;
            while ((ligne=in.readLine()) != null) {
                System.out.println(ligne);
            }
        } finally {
            if (in != null) {
                in.close();
            }
        }

        // Ecriture
        PrintWriter out = null;
        try {
            out = new PrintWriter(new FileWriter("fichier.txt"));
            out.println("Hello World");
            out.println(42);
        } finally {
            if (out != null) {
                out.close();
            }
        }
    }
}
```

2.2 Traitement des flux d'octets

Le mécanisme de *serialisation* permet de transformer un objet dans un flux d'octets. Pour l'utiliser, la classe de tel objet doit implémenter l'interface `java.io.Serializable`. Cette interface ne nécessite pas de redéfinir une quelconque méthode mais il est nécessaire de définir un attribut **serialVersionUID** permettant d'identifier la classe.

La classe `java.io.FileInputStream` est une classe d'extraction qui permet de construire un flux d'octets d'un programme vers un fichier. La classe `java.io.DataInputStream` permet de lire les données primitives d'un objet dans un flux de données binaires. La classe `java.io.ObjectInputStream` permet de lire directement l'objet dans le flux, avec tous ses champs.

Il existe deux méthodes pour écrire un objet dans le flux : `writeObject()` et `writeUnshared()`. La différence entre les deux est que `writeObject` écrit les objets de manière unique ; autrement dit, un même objet

ne sera inscrit qu'une seule fois dans le flux de données. Par contre, `writeUnshared` écrit les objets dans le flux qu'ils aient été écrits précédemment ou non.

À noter que ces méthodes ne fonctionnent que si les champs des objets sont des types primitifs. Si la conversion d'un objet en données binaires n'est pas trivial, il est nécessaire de définir les méthodes `writeObject` et `readObject`.

Exemple : Considérons le code pour la lecture et écriture des instances d'une classe Animal dans des fichiers binaires.

```
import java.io.*;
class Animal implements Serializable {
    private static final long serialVersionUID = 4L;
    private String name;
    ...
}

public class Application {
    public static void main(String[] args) throws IOException
    {
        // Lecture
        ObjectInputStream ois = new ObjectInputStream(
                                    new FileInputStream("objets")
                                );
        try {
            while (true) {
                System.out.println(ois.readObject().toString());
            }
        } catch (ClassNotFoundException cnf) {
            // erreur de lecture
        } catch (EOFException eof) {
            // fin de fichier
        }
        ois.close();

        // Ecriture
        try {
            ObjectOutputStream oos = new ObjectOutputStream(
                                        new FileOutputStream("objets")
                                    );
            oos.writeObject(new Animal(...));
            oos.close();
        } catch (Exception e) {
            e.getStackTrace();
        }
    }
}
```

3 Travail à faire

3.1 Exercice 1

Écrire, en utilisant la classe File (dans `java.io`), un programme qui prend en paramètre un chemin d'accès et qui teste tout d'abord si ce chemin existe et s'il est un fichier ou un répertoire. Si c'est un fichier, afficher sa taille et son chemin absolu. Si c'est un répertoire, afficher la liste des fichiers et répertoires qu'il contient.

Voici quelques exemples d'exécution :

```
Chemin: README.md
Type: Fichier
Chemin absolu: /home/etud/aoop/tp6/README.md
Taille: 135 octets
```

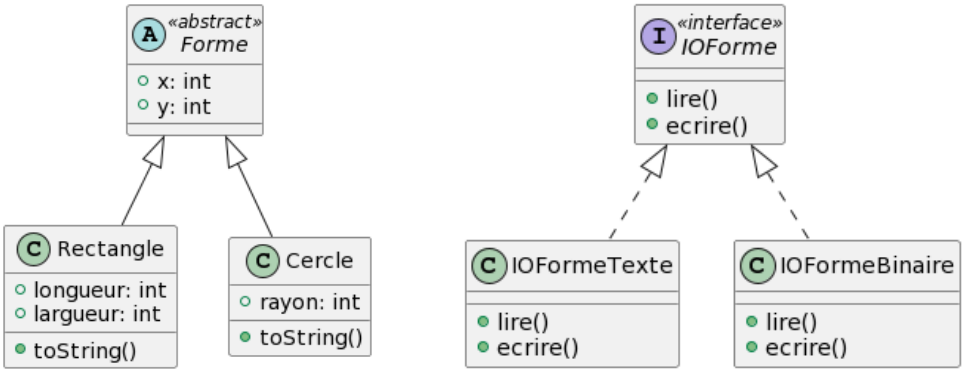
```
Chemin : ../../../unfichier.txt
Type: Fichier
Chemin absolu: /home/etud/unfichier.txt
Taille  : 7947 octets
```

```
Chemin : .
Type: Repertoire
Liste des fichiers et répertoires du dossier "." :
  README.md
  lib
  src
  bin
  .vscode
```

3.2 Exercice 2

Parmi les fonctions d'un outil de manipulation des images (par exemple, GIMP ou Inkscape) se trouve la manipulation de fichiers avec l'information sur les images à traiter. Dans ce contexte, on souhaite implémenter une application avec des méthodes de lecture et d'écriture pour deux formes géométriques : rectangles et cercles (Figure 1a).

La gestion des entrées et sorties est gérée par l'interface `IOForme` (voir code ci-dessous), qui comporte les méthodes `lire` et `sauver`. En plus, il faut considérer deux types de fichiers : texte et binaires (Figure 1b).



(a) Classes pour les formes géométriques

(b) I/O

```java
import java.io.IOException;
import java.util.Collection;

public interface IOForme {
    public Collection<Forme> lire(String filename)
        throws IOException;
    public void sauver(Collection<Forme> formes, String filename)
        throws IOException;
}
```

1. Implémentez la classe abstraite `Forme` et les sous-classes `Rectangle` et `Cercle`.
2. Implémentez l'affichage d'une collection de formes (à vous de déterminer combien des formes ajouter) de la manière suivante :

   ```
   rectangle 3 2 5 6
   cercle 3 0 5
   cercle 3 3 6
   ...
   ```

3. Implémentez l'interface `IOForme`.

4. Créez la classe `IOFormeTexte` pour lire et écrire des fichiers texte qui utilisent le format : `<type> <attributes>` (pareil au format d'affichage demandé dans la question 2).
5. Implémentez la méthode de lecture d'un fichier contenant de formes. Cette méthode doit retourner une collection de formes.
6. Implémentez l'écriture d'un fichier texte a partir d'une collection de formes.
7. Créez la classe `IOFormeBinaire` pour lire et écrire des fichiers binaires.
8. Implémentez les méthodes de lecture et écriture des fichiers binaires.
   Note : N'oubliez pas la serialisation de l'objet avant son écriture.

Références

[1] Package java.io (Java Platform SE 8),
https://docs.oracle.com/javase/8/docs/api/java/io/package-summary.html

[2] J.M. Doudoux - 13. Les flux d'entrée/sortie,
https://www.jmdoudoux.fr/java/dej/chap-flux.htm

[3] Java I/O Tutorial,
https://www.javatpoint.com/java-io

[4] www.falkhausen.de - Enhanced Java class diagrams
https://www.falkhausen.de/Java-8/java.io/index.html